```
/grokit$ find ./ -name "*"
./
./tools
./tools/opengrok-0.12.1.tar.gz
./tools/exuberant-ctags.tar
./tools/jre-7u65-linux-x64.tar.gz
./tools/apache-tomcat-8.0.8.tar.gz
./grokit
./README.txt
```

Figure 6 : List of files in the grokit bundle for Linux

```
/grokit$ ls grokit_files
apache-tomcat-8.0.8  ctags58  jre1.7.0_65  opengrok-0.12.1  projects
/grokit$ ls grokit_files/projects/
ffmpeg-2.5  p7zip 9.20.1
/grokit$ ls grokit files/projects/ffmpeg-2.5/
data  etc  ffmpeg-2.5.war  modified_war
```

Figure 7: List of directories created

A project directory is also created inside *grokit_files*. For every project whose cross-reference is created, a directory is created inside the project directory. In the above example, two projects are cross-referenced *(ffmpeg-2.5* and *p7zip_9.20.1)*. All the cross-reference information generated by Opengrok is stored inside the respective project directories. These projects can be accessed from the browser using the URL *http://localhost:8080/ffmpeg-2.5* and *http://localhost:8080/p7zip_9.20.1*, respectively (replace localhost with the appropriate IP address if not accessed on localhost).

For those interested in exploring the source, grokit is written in Python. The entire source code for grokit can be obtained from *https://github.com/raghusesha/grokit*. Apart from the standard Python libraries, it only needs *BeautifulSoup* (and the *lxml* backend) for editing the XML files. All the core functionality is present in *utils.py.* In case you want to use different versions of the tools (*apache-tomcat, opengrok, exuberant-ctags* or *jre*), you will have to copy these in the *tools* directory and edit *platdefines.py* accordingly.

Supported platforms and dependencies

Currently, grokit binaries are available for Windows 32-bit and Linux 64-bit on the grokit home page. However, the entire source code is available at *https://github.com/raghuesha/grokit* and can be used on any platform.

grokit binaries for Linux are built on Ubuntu 12.04, which uses *glibc 2.15* by default. So, these may not work on platforms with earlier versions of *glibc* and may need recompilation from source.

The future

grokit can be used to cross-reference any source code. It has been successfully used to cross-reference Linux, Android, ffmpeg and other open source projects. Each cross-referenced project is accessible at a separate URL. So, it is extremely straightforward to create a set-up similar to *http://androidxref.com*, in which different projects (in this case, different versions of Android) are available at unique URLs using grokit.

Companies can use grokit to cross-reference their source code on internal servers, and let their engineers access the code within the firm's Intranet without any security concerns. However, if a company wants to provide access to its cross-referenced code over a secure channel (say HTTPS) with appropriate authentication, a substantial framework needs to be created around grokit. This will be considered for a future column. END

References

[1] *http://blog.vinceliu.com/2008/06/installing-opengrok-on-ubuntu-linux.html*
[2] *https://www.python.org/doc/*

By: Raghu Sesha Iyengar

The author is working with AgreeYa Mobility as senior engineering manager. He has over 10 years of experience working on mobile software, especially Linux device drivers for Android devices. He can be reached at *raghu.sesha@gmail.com* or *raghu.iyengar@agreeyamobility.net*

...Continued from page 59

- Each module has two main files— *config/module.config.php* for module-specific configurations, and *Module.php*, executes first when any action in the module is called.
- *src:* This directory contains *Controller* and other module files.
- *view:* This directory contains the view part of a module. In ZF2, each action has specific view files with a 'phtml' extension. A module-specific layout is also listed in this directory. Figure 6 shows the directory structure of the module directory.

- **Public:** This directory contains all publicly accessible content like *css, html, js* and *.htaccess* files as shown in Figure 7.
- **Vendor:** This directory contains libraries that are used in the project as shown in Figure 8.

This is the typical directory structure of ZF2. The Zend Framework is loose coupled and very flexible. So, you can change the path of the vendor, module and data within your server, and all paths are configurable in the *application.config.php* file.

Note: The sample code for developing a 'To-do project' is hosted on Github 'ZF2TodoOsfy' - *https://github.com/savankoradia/ZF2TodoOsfy* and will be updated with each tutorial.

END

References

[1] *http://en.wikipedia.org/wiki/Zend_Framework*
[2] *https://github.com/zendframework/ZendSkeletonApplication*

By: Savan Koradia

The author works as a senior PHP Web developer at Multidots Solutions Pvt Ltd. In his spare time he writes tutorials and teaches other developers to write better and more secure code in their projects. You can contact him at *savan.koradia@multidots.in* or on Skype: *savan.koradia.multidots*